Testing Plan for FSM Software

Table of Contents

1. **Testing Objectives and Scope**
    - Core Functionalities
    - Compliance and Security Standards
    - System Integration and Data Synchronization
    - Reliability and Scalability
    - Scope Overview
2. **Testing Guidelines**
    - Role-Based Testing
    - Automated and Integration Testing
    - Compliance and Security Verification
    - Usability and Accessibility Testing
    - Data Synchronization and Offline Testing
    - Risk-Based and Performance Validation
3. **Testing Methodologies**
    - Exploratory and Regression Testing
    - Acceptance and User Scenario Testing
    - Automated API Testing
    - Continuous Integration Pipeline Testing
4. **Compliance and Security Deep Dive**
    - OSHA, DOT, CCPA, and PCI-DSS Compliance
    - Security Testing: Penetration and Vulnerability Scanning
    - Data Encryption, Incident Response, and Risk Management
    - Compliance and Security Reporting
5. **Test Environment Setup**
    - Infrastructure Components

- Software and Network Configurations
- Test Data and Security Setup
- Automation and CI/CD Tools

6. **Test Case Design**
   - Key Test Cases Overview
7. **Reporting and Documentation**
   - Centralized Reporting
   - Documentation Archive
   - Stakeholder Communication

1. Testing Objectives and Scope

1.1. Testing Objectives

1.1.1. Core Functionalities:

- User Role Management: Verify role-based access control and secure authentication using DUO for different roles (Technicians, Managers, Administrators).
- Job Scheduling and Dispatch: Confirm that job scheduling, technician dispatching, and routing are effective, conflict-free, and up to date with real-time notifications.
- GPS Tracking and Route Optimization: Ensure real-time tracking accuracy and optimized routing via Google Maps API and Mapbox.
- Inventory Management: Validate inventory tracking, stock alerts, and IoT-supported equipment monitoring.
- Invoicing and Payment Processing: Verify invoicing accuracy and secure payment processing, including integration with Stripe and PayPal.
- Customer Self-Service: Test customer portal features for appointment management, service history, and payments.

1.1.2. Compliance and Security Standards:

- Regulatory Compliance: Confirm adherence to OSHA, DOT, CCPA, and PCI-DSS standards, ensuring functionalities meet these regulatory guidelines.

- Data Security and Privacy: Verify robust data protection using AES-256 for stored data and TLS for data in transit, ensuring secure access control across FSM modules.

1.1.3. System Integration and Data Synchronization:

- External System Integrations: Test seamless API integration and data synchronization with CRM, ERP, GPS, MFA, and financial systems like QuickBooks, Stripe, and PayPal.
- Data Consistency: Verify accurate data handling and synchronization, especially when switching between offline and online states for field technicians.

1.1.4. Reliability and Scalability:

- Performance and Uptime: Ensure the system meets a 99.9% uptime target and performs reliably under increased user loads.
- Scalability for Growth: Test for scalability, ensuring FSM can support future enhancements (e.g., IoT integrations).
- Usability and Accessibility: Validate that interfaces are user-friendly and accessible, meeting Section 508 and other accessibility standards.

1.2. Scope

1.2.1. Functional Testing:

- Core FSM functionalities, including role management, scheduling, dispatching, GPS tracking, inventory management, invoicing, and customer engagement.

1.2.2. Integration Testing:

- API and middleware integration testing with CRM, ERP, GPS, payment processing, and DUO Security systems.

1.2.3. Non-Functional Testing:

- Performance and Scalability: Validate response times, throughput, and reliability under various conditions.
- Compliance and Security: Verify compliance with OSHA, DOT, CCPA, and PCI-DSS standards, and ensure data encryption and privacy.
- Usability: Confirm ease of use and accessibility, especially for mobile and offline functionalities.

1.2.4. Risk-Based Testing:

- Prioritize testing high-impact areas such as scheduling, GPS tracking, payment processing, and data synchronization, addressing scenarios like network outages and payment gateway failures.

1.2.5. System Reliability and Maintenance:

- Assess fault tolerance, backup mechanisms, and recovery processes to meet uptime requirements and support ongoing maintenance.

2. Testing Guidelines

2.1. Role-Based Testing Strategy:

- Design and execute tests to verify functionality for each role (Technician, Manager, Administrator), ensuring appropriate permissions and data access based on role.
- Conduct security tests to verify that restricted data or features remain inaccessible to unauthorized roles.

2.2. Automated Testing for High-Impact Scenarios:

- Prioritize automation for high-frequency and repetitive tests, including login with DUO MFA, core CRUD operations, and regression tests.
- Integrate regression testing within the CI pipeline to ensure continuous validation of critical features after updates or code changes.

2.3. Integration Testing with External Systems:

- Test interactions with external systems (e.g., CRM, ERP, Google Maps, payment gateways) to confirm stable, accurate integrations.
- Implement failover scenarios and error handling for critical services like GPS tracking and payment processing, ensuring reliable performance under various conditions.

2.4. Compliance and Security Verification:

- Perform compliance checks for OSHA, DOT, CCPA, and PCI-DSS standards, particularly for job scheduling, data encryption, and access control.
- Conduct security testing for role-based access, secure data handling, and MFA functionality. Use encryption validation tests to confirm data security for sensitive fields.

2.5. User Experience and Accessibility:

- Validate UI consistency and intuitiveness for different user roles, ensuring easy navigation and task completion.
- Conduct accessibility testing to ensure compliance with standards like Section 508, improving usability for individuals with disabilities.

2.6. Data Synchronization and Offline Capability Testing:

- Simulate offline scenarios to verify FSM's offline functionalities, ensuring technicians can access and update job information without connectivity.
- Test data synchronization processes upon reconnection, confirming that offline updates are seamlessly integrated into the central database.

2.7. Risk-Based Testing Approach:

- Identify and prioritize high-risk areas for testing, such as job scheduling, GPS tracking, and payment processing.
- For each high-risk area, design tests that simulate potential risk scenarios (e.g., payment failures, GPS inaccuracies) to assess the system's resilience.

2.8. Performance and Scalability Validation:

- Conduct load tests to validate the system's performance under expected and peak user loads, particularly for real-time tracking and scheduling functions.
- Ensure the system meets response time and uptime requirements, focusing on scalability for future growth.

2.9. Reporting and Documentation Standards:

- Maintain a centralized testing record that documents each test case's status, identified bugs, and resolution progress.
- Use standardized report templates to communicate test results, including compliance findings, weekly summaries, and final test completion reports.

3. Testing Methodologies

3.1. Exploratory Testing:

- **Purpose**: Enables testers to dynamically explore the FSM system, uncovering edge cases and unexpected behaviors.

- **Execution**:
  - Schedule exploratory sessions after completing initial functional tests.
  - Focus on complex workflows, like multi-role access and simultaneous user actions.
- **Documentation**: Log observations, potential issues, and any unusual behaviors. Use insights to enhance future test cases or identify new edge cases.

3.2. Regression Testing:

- **Purpose**: Ensures new code changes do not disrupt existing functionalities, essential after each update or bug fix.
- **Execution**:
  - Automate regression tests for core features like login, scheduling, and role-based access.
  - Integrate into the CI pipeline to run automatically with each new build.
- **Documentation**: Record results in the regression test suite, updating any modified tests as features evolve.

3.3. Acceptance Criteria Testing:

- **Purpose**: Validates that each module aligns with user acceptance criteria, ensuring software meets end-user expectations.
- **Execution**:
  - Develop test cases based on user stories or requirement documentation.
  - Conduct acceptance testing with end-users or stakeholders to verify satisfaction.
- **Documentation**: Document acceptance outcomes, tracking any unmet criteria for further adjustments or revisions.

3.4. User Scenario Testing:

- **Purpose**: Tests end-to-end workflows to confirm that key user journeys (e.g., technician job acceptance to invoicing) are smooth and functional.
- **Execution**:
  - Define critical workflows, detailing each user interaction from start to finish.
  - Simulate these workflows for different roles to validate cross-role dependencies.

- **Documentation**: Log results for each scenario, noting any usability improvements or issues affecting workflow continuity.

3.5. Automated API Testing:

- **Purpose**: Verifies reliable data exchange and response times in FSM's integrations with third-party systems.
- **Execution**:
  - Use tools like Postman or JUnit to automate API tests, focusing on data synchronization and error handling for CRM, ERP, GPS, and payment gateways.
  - Test each API endpoint for expected responses, latency, and fault tolerance.
- **Documentation**: Maintain API test logs, recording response times and error handling results to ensure consistent integration reliability.

3.6. Continuous Integration (CI) Pipeline Integration:

- **Purpose**: Automates testing within the CI pipeline, providing continuous feedback on code quality and stability.
- **Execution**:
  - Configure the CI environment (e.g., Jenkins, GitHub Actions) to trigger unit, regression, and API tests upon each code commit.
  - Review CI test results regularly to ensure prompt identification and resolution of issues.
- **Documentation**: Track CI logs for test execution history, analyzing any recurring issues introduced by recent code changes.

4. Compliance and Security Deep Dive

4.1. Compliance Testing Plan

4.1.1. OSHA and DOT Compliance Testing

- **Scope**: Confirm that job scheduling, routing, and technician workloads align with OSHA and DOT requirements for safety and operational efficiency.
- **Execution**:

    - **Job Assignment**: Validate that workload limits prevent over-scheduling, ensuring technician rest periods meet OSHA guidelines.
    - **Route Validation**: Confirm that GPS routes avoid restricted zones and comply with DOT standards.
- **Documentation**:
    - Use a **Compliance Checklist** for OSHA and DOT, detailing each compliance aspect and any deviations.
    - Report compliance deviations for necessary adjustments and remediation.

4.1.2. CCPA Compliance Testing

- **Scope**: Validate FSM software's adherence to CCPA, ensuring that users can control personal data access and deletion.
- **Execution**:
    - **Data Access and Deletion**: Test options allowing users to access, delete, or export their personal data in line with CCPA requirements.
    - **Data Minimization**: Confirm that only essential data is collected and securely stored with AES-256 encryption.
- **Documentation**:
    - Maintain a **CCPA Compliance Report**, covering privacy-related test cases, data access, deletion options, and encryption.
    - Conduct periodic **Compliance Audits** to ensure consistent adherence to CCPA requirements.

4.1.3. PCI-DSS Compliance Testing

- **Scope**: Ensure secure handling of payment data per PCI-DSS standards, covering encryption, data masking, and access control.
- **Execution**:
    - **Encryption and Masking**: Validate that payment data is masked in the UI and encrypted during transmission.
    - **Access Control**: Restrict access to payment processing features, limiting it to authorized roles.
- **Documentation**:
    - Maintain a **PCI-DSS Compliance Checklist**, detailing encryption methods, access controls, and masking protocols.

- Conduct a **Monthly Payment Security Review** to verify PCI-DSS compliance consistency.

4.2. Security Testing Plan

4.2.1. Penetration Testing

- **Scope**: Identify and assess vulnerabilities through simulated attacks on sensitive data areas, such as authentication and payment systems.
- **Execution**:
  - **External Penetration Testing**: Simulate attacks targeting APIs, public interfaces, and payment processing from outside the network.
  - **Internal Penetration Testing**: Test internal system permissions, focusing on potential vulnerabilities in user role access and data storage.
- **Tools**: Use **OWASP ZAP**, **Burp Suite**, and **Nessus** for penetration testing.
- **Documentation**:
  - Generate a **Penetration Test Report** with identified vulnerabilities, their severity, and recommended remediations.
  - Follow a **Vulnerability Management Process** to prioritize and resolve high-risk issues.

4.2.2. Vulnerability Scanning and Security Audits

- **Scope**: Continuously monitor FSM software for vulnerabilities, especially after deployment.
- **Execution**:
  - Schedule **Automated Vulnerability Scans** (weekly or bi-weekly) to detect codebase weaknesses.
  - Conduct **Quarterly Security Audits** to assess new vulnerabilities in updated software, third-party integrations, and code changes.
- **Tools**: Use **Qualys**, **Nessus**, or **Aqua Security** for regular scanning.
- **Documentation**:
  - Maintain **Vulnerability Scan Reports**, detailing scan results and remediation actions.
  - Keep an **Audit Log** of security checks, ensuring compliance and transparency.

4.2.3. Data Encryption and Privacy Testing

- **Scope**: Validate that sensitive data is encrypted and secure, both at rest and in transit.

- **Execution**:
  - **Data at Rest**: Confirm that sensitive data on servers is encrypted using AES-256.
  - **Data in Transit**: Validate TLS 1.2/1.3 protocols for data exchanges between client and server.
- **Tools**: Use **OpenSSL** for TLS testing and **GnuPG** for encryption validation.
- **Documentation**:
  - Log results in a **Data Encryption Report**, noting encryption standards, algorithms, and security configurations.
  - Regularly review encryption practices to ensure they align with evolving security standards.

4.3. Incident Response and Risk Mitigation

4.3.1. Incident Response Plan (IRP)

- **Scope**: Outline procedures to effectively manage and respond to security incidents, minimizing impact and ensuring a structured response.
- **Components**:
  - **Preparation**: Define roles and responsibilities for response actions and ensure team readiness.
  - **Identification and Containment**: Establish criteria to identify incidents and execute containment protocols.
  - **Remediation and Recovery**: Set procedures for root cause analysis, data recovery, and securing vulnerabilities.
- **Documentation**:
  - Maintain an **Incident Log** recording each incident, response actions, and resolutions.
  - Conduct **Post-Incident Reviews** to capture lessons learned and improve security practices.

4.3.2. Risk Assessment and Management

- **Scope**: Identify and prioritize high-risk areas in FSM software to mitigate potential threats.
- **Execution**:
  - **Threat Modeling**: Perform risk assessments to identify vulnerabilities, focusing on high-risk areas like authentication and data access.

- **Risk Assessment Matrix**: Use a matrix to classify risks by severity and impact, guiding focused resource allocation.

- **Documentation**:
  - Document findings in a **Risk Assessment Report**, detailing each risk, mitigation strategy, and follow-up action.
  - Maintain a **Risk Management Log** updated with new risks and mitigations.

4.4. Compliance and Security Reporting

For accountability and adherence to standards, generate the following reports and maintain them in the documentation archive:

1. **Compliance Checklist**: Covers OSHA, DOT, CCPA, and PCI-DSS compliance, recording each requirement's adherence.
2. **Security Test Reports**: Periodic reports detailing outcomes from penetration tests, vulnerability scans, and encryption tests.
3. **Incident Logs**: Detailed records of security incidents, response actions, and outcomes.
4. **Risk Management Log**: Documentation of identified risks, assessments, and resolutions.
5. **Quarterly Compliance Review**: Summarizes ongoing compliance adherence, identifying areas for improvement or additional focus.

5. Test Environment Setup

5.1. Infrastructure Components

5.1.1. Servers and Hardware:

- **Web Server**: A dedicated server to host the FSM application backend, running Java Spring Boot to manage business logic and data integrations.
- **Database Server**: PostgreSQL or a similar relational database to store essential data, including job records, user roles, inventory, and transactions.
- **Mobile Devices**: iOS and Android devices with GPS capabilities to test real-time tracking, offline functionalities, and mobile UI usability.

- **Peripheral Devices**: Barcode scanners for inventory management, GPS devices for field tracking, and optional receipt printers for invoicing.

5.2. Software Requirements

5.2.1. Operating Systems:

- **Desktop**: Windows and macOS for web access, administrative functions, and backend management.
- **Mobile**: iOS and Android for testing mobile functionalities like GPS tracking, job updates, and customer interactions.

5.2.2. Application Frameworks:

- **Backend**: Java Spring Boot for backend processes, business logic, and integration management.
- **Frontend**: React or Angular for the FSM dashboard and mobile app, optimized for cross-platform compatibility on both desktop and mobile devices.

5.2.3. Third-Party Integrations:

- **Authentication**: DUO Security or OAuth 2.0 for multi-factor authentication and role-based access control.
- **Payment Gateways**: Active Stripe and PayPal accounts for validating invoicing, payment processing, and refund functionalities.
- **Mapping and GPS Services**: Google Maps API and Mapbox for location tracking, route optimization, and geofencing.
- **ERP/CRM Systems**: Test integrations with QuickBooks and Salesforce for data synchronization and accurate record-keeping.

5.3. Network and Connectivity Configurations

5.3.1. Standard Network Configuration:

- Use a high-speed internet connection to support real-time API calls and interactions with third-party services like Google Maps, DUO, and payment gateways.

5.3.2. Network Conditions Testing:

- **Offline Mode**: Set up controlled disconnections to test FSM offline capabilities, especially for job updates, inventory tracking, and data synchronization upon reconnection.
- **Variable Network Speeds**: Simulate low-bandwidth conditions to evaluate app performance under rural or remote field conditions.

5.4. Test Data and Accounts Setup

5.4.1. User Accounts:

- **Technician**: Limited access for tasks like job updates, GPS tracking, and inventory management.
- **Manager**: Access to scheduling, dispatching, and reporting features.
- **Administrator**: Full access to system management, role assignments, and compliance features.

5.4.2. Sample Data:

- **Job Records**: A mix of job types, statuses, and technician assignments to test scheduling, dispatching, and completion workflows.
- **Inventory**: Test inventory records with quantities, usage rates, and stock alerts to validate inventory tracking and management.
- **Customer Data**: Mock customer records for validating appointment scheduling, service history access, and invoicing.
- **Performance Metrics**: Sample data for performance analysis, such as scheduling efficiency scores and technician KPIs, used for performance reporting tests.

5.5. Security and Compliance Configurations

5.5.1. Data Security:

- **Encryption**: Enable AES-256 encryption for all sensitive data stored in the database and use TLS 1.2/1.3 for secure data transmission.
- **Access Control Policies**: Implement role-based access control configurations, ensuring that technicians, managers, and administrators can access only permitted data and functionalities.

5.5.2. Compliance Testing Setup:

- **OSHA and DOT Flags**: Configure compliance flags within scheduling and routing features to simulate OSHA and DOT requirements.
- **Data Retention and Deletion Rules**: Set up data retention policies to support CCPA compliance, ensuring data deletion upon user requests and following specific retention periods.

5.6. Test Automation and CI/CD Tools

5.6.1. Automation Frameworks:

- **Selenium**: For automated browser testing across different web interfaces and validating role-based access control.
- **Appium**: For mobile test automation, ensuring consistent UI behavior, GPS tracking, and offline features.
- **JUnit/TestNG**: For backend unit tests, covering job scheduling logic, data handling, and compliance checks.
- **Load Testing Tools**: JMeter or Locust for simulating network and user loads to validate system scalability and performance.

5.6.2. Continuous Integration (CI) Setup:

- **CI Tools**: Jenkins or GitHub Actions for automated test execution on each new build or code commit.
- **Pipeline Configuration**: Configure CI to automatically run regression, unit, and API tests after code changes, enabling continuous testing and feedback.

6. Test Case Design

Each test case will include:

Label	Description
Test Case ID	Unique identifier for the test case
Description	Brief overview of the test case and what it validates
Preconditions	Initial conditions required before executing the test
Test Steps	Step-by-step instructions to execute the test
Expected Results	Expected outcome if the test is successful
Compliance Checks	Relevant regulatory or compliance standards (e.g., OSHA, CCPA)

6.1. Test Cases

Test Case ID	Description	Preconditions	Test Steps	Expected Results	Compliance Checks
TC-01	Validate secure user authentication with DUO MFA and role-based access control	DUO Security MFA set up; sample users for Technician, Manager, and Administrator roles	1. Log in as Technician with MFA. 2. Log in as Manager with MFA. 3. Log in as Administrator with MFA. 4. Attempt restricted feature access for each role.	Each role can only access permitted features, and MFA works as expected.	CCPA (data security), Role-Based Access
TC-02	Verify job scheduling based on technician availability, skill set, and proximity	Database populated with job records and technician profiles (skills, location, availability)	1. Schedule a job for a technician who is available and has the required skills. 2. Attempt scheduling a job for a busy technician. 3. Schedule a job and verify technician notifications.	Job scheduling respects technician availability and skill set; notifications sent.	OSHA (workload compliance), CCPA (privacy)
TC-03	Validate real-time GPS tracking and route optimization	Technician assigned a job with GPS-enabled device; internet connectivity	1. Begin real-time tracking of technician location. 2. Use route optimization to suggest the most efficient route. 3. Check for geofence alerts upon route deviation.	Technician location updates in real time; route suggestions are optimal; geofence alerts on deviation.	DOT (route compliance)
TC-04	Validate inventory level tracking and stock alert functionality	Sample inventory records in database, including low-stock items	1. Access inventory management as a technician. 2. Update stock after job completion to reflect used items. 3. Trigger low-stock alert below threshold.	Stock updates accurately; low-stock alert triggers; data syncs.	CCPA (data protection), OSHA (safety)

TC-05	Verify invoicing and payment processing accuracy through Stripe/PayPal	Job marked as completed with associated costs; active Stripe/PayPal account	1. Generate invoice for completed job. 2. Process payment via Stripe. 3. Process payment via PayPal. 4. Test partial payment and refund scenarios.	Accurate invoice generation; successful payments; proper handling of refunds.	CCPA (payment data protection), PCI-DSS
TC-06	Test customer portal functionalities for appointment management, service history, and payment	Sample customer records in the system	1. Log into customer portal. 2. Schedule an appointment. 3. Access service history. 4. Complete payment for an appointment.	Customer can manage appointments, view service history, and make payments successfully.	CCPA (data privacy), PCI-DSS
TC-07	Verify offline data access for field technicians and post-reconnection synchronization	Technician with an assigned job goes offline	1. Access job details and update status offline. 2. Reconnect and sync data with main database.	Offline data access consistent; data syncs correctly upon reconnection.	CCPA (data handling and sync)
TC-08	Verify OSHA and DOT compliance for technician safety and routing	Compliance rules configured in scheduling and routing modules	1. Attempt job assignment that breaches OSHA standards (e.g., workload limits). 2. Test route suggestions against DOT standards.	System blocks actions that breach OSHA/DOT, displaying appropriate alerts.	OSHA, DOT compliance
TC-09	Assess system performance under high user loads and data transactions	Load testing tool (e.g., JMeter) configured with concurrent user scenarios	1. Simulate 500+ concurrent users logging in, accessing scheduling, and using GPS tracking. 2. Measure response times and error rates at peak.	Response times within limits; low error rates under peak usage conditions.	Performance SLAs (99.9% uptime)

7. Reporting and Documentation

7.1. Centralized Reporting Process

7.1.1. Daily Test Execution Report:

- **Purpose**: Provides real-time progress updates and tracks daily test outcomes.
- **Content**: Summary of tests executed, passed, failed, and any new defects identified.
- **Audience**: QA team, test leads.
- **Frequency**: Daily.

7.1.2. Bug Report and Tracking:

- **Purpose**: Records identified issues, their severity, and resolution status.
- **Content**:
  - **Bug ID**: Unique identifier.
  - **Description**: Brief overview of the issue.
  - **Steps to Reproduce**: Details for replicating the bug.
  - **Severity and Priority**: Assessment of the issue's impact and urgency.
  - **Status**: Open, In Progress, Resolved, Closed.
- **Audience**: QA team, developers.
- **Tool**: JIRA, Bugzilla, or a similar bug-tracking system.
- **Frequency**: Ongoing as issues are identified.

7.1.3. Weekly Test Status Report:

- **Purpose**: Summarizes weekly testing progress, highlighting key issues and focus areas for the coming week.
- **Content**:
  - Overall test progress (e.g., % completed).
  - Critical bugs and open issues.
  - Upcoming focus areas.
- **Audience**: QA leads, project stakeholders.
- **Frequency**: Weekly.

7.1.4. Compliance and Security Review Report:

- **Purpose**: Tracks security and compliance testing results for OSHA, DOT, CCPA, and PCI-DSS standards.
- **Content**:
  - Compliance checklist results.
  - Security testing summary (e.g., penetration testing, vulnerability scan outcomes).
  - Compliance gaps or risks identified.
- **Audience**: Project stakeholders, compliance team.
- **Frequency**: Monthly or at major testing milestones.

7.1.5. Final Test Summary Report:

- **Purpose**: Provides a comprehensive overview of all testing phases and outcomes, serving as a formal assessment of the software's readiness.
- **Content**:
  - Final test coverage summary (total tests executed, passed, and failed).
  - Defect summary (categorized by severity, with resolution status).
  - Compliance confirmation (OSHA, DOT, CCPA, PCI-DSS).
  - Performance and scalability test results.
  - Recommendations for release or further testing.
- **Audience**: Project stakeholders, QA team, management.
- **Frequency**: Upon test phase completion.

7.2. Documentation Archive and Knowledge Base

To support future references and project transparency, testing documents and records will be stored in a centralized knowledge base. This archive includes:

- **Testing Plans**: All finalized test plans, objectives, and scope.
- **Execution Logs**: Records of daily and weekly reports, including execution results for each test case.
- **Bug Reports**: Comprehensive bug tracking logs with resolutions.
- **Compliance Records**: Detailed compliance testing logs and security audits.
- **Performance Metrics**: Records of response times, load testing, and scalability benchmarks.

Storage Location: All documents are stored in a centralized repository (e.g., SharePoint, Confluence) with access controls for authorized personnel.

7.3. Stakeholder Communication

Regular communication ensures that stakeholders are updated on testing progress, compliance findings, and any critical issues that may impact timelines.

- **Progress Updates**: Share summarized progress at key milestones, particularly for high-priority or compliance-related tests.
- **Critical Defects Notifications**: Immediate reporting of high-severity issues that impact the project timeline or functionality.
- **Compliance Checkpoints**: Share updates on major compliance testing phases, especially related to OSHA, DOT, and CCPA standards.

Format: Reports can be shared through project management tools, periodic email updates, or during scheduled stakeholder meetings.